

Institutional Memory

19 engines become 1 product. The customer never sees versions. They ask "What have we learned?"

Baseline	Commit ad00d31. V5 complete (8 specs). V6 complete (6 specs, 2 wiring gaps). 74 backend modules, 28 frontend files. 19 cognitive engines. 423 tests pass.
V8 shift	V6 improved the organization. V7 revised Maestro's theory. V8 unifies them: 19 engines become 1 product — Institutional Memory. The customer never sees "DNA" or "Institution Model" or "Trajectory Intervention." They see "What we've learned."
Constitutional law	"Maestro should become the living institutional memory of an organization — continuously observing, questioning, revising, and deepening its understanding until it becomes the most accurate model of how that organization thinks, decides, and learns."
Deliverable	Step 0: fix 2 V6 wiring gaps (1.5 hours). Then 6 V8 specs (~12 days): Institutional Memory Timeline, Institutional Confidence, Unknown Unknowns, Organizational Curiosity, Understanding Map, Unified Surface.

WHY V8 EXISTS

V3 observed. V4 judged. V5 disappeared. V6 adapted. V7 revised its theory. Each version added engines. 19 cognitive engines across 6 constitutions. All invisible. All enhancing existing surfaces. That is a significant engineering achievement.

V8 is different. V8 says: stop producing engines. Produce Institutional Memory. The customer should never see "DNA," "Institution Model," "Trajectory Intervention," "Compounding Judgment," or "Closed Learning Loop." They should ask "Tell me what we've learned" and hear: "Over the last six months your organization quietly stopped depending on one engineer, reduced approval latency by 31%, shifted from reactive reviews to planned reviews, and now predicts delivery with 18% higher accuracy." That is infinitely more valuable than 19 engine names.

The 19 engines remain internally. V8 does not delete them. It changes how they are EXPOSED. Internally: 19 modules doing their work. Externally: 1 product called Institutional Memory that answers "What has this organization learned?"

Same discipline. Every spec has: principle, codebase gap, files, API, acceptance test, effort, dependencies. The V5 litmus test (UI simpler) and V6 litmus test (permanently improves) are retained. A new V8 litmus test is added: "Does this make the customer say 'Maestro knows how our company works'?" If yes, it is V8.

Step 0 — Fix 2 V6 Wiring Gaps (MUST DO FIRST, 1.5 hours)

2 GAPS FROM ROUND 24 — FIX BEFORE V8

Gap 1: Background loop NOT hooked into `live_ingest()` (30 min). Background adaptation only runs when TODAY fetches the API — not on signal ingest. V6 Law 2 violated. Fix: add `BackgroundLoop.check()` call at end of `oem_state.py live_ingest()`.

Gap 2: DNA NOT referenced in `wisdom.py` (1 hour). DNA produces 7 chromosomes but does not filter recommendations. Fix: modify `wisdom.py` to import `organizational_dna.py`, compute `alignment_score` per recommendation.

These are 1.5 hours total. Do them first. Then build V8.

The 6 V8 Specifications

#	Specification	Effort	What It Replaces/Creates
1	Institutional Memory Timeline	2 days	Replaces Evolution Narrative (autobiography). Experiential moments, not literary chapters.
2	Institutional Confidence	1.5 days	NEW. "How well do we understand Finance? 96%. Sales? 18%." Coverage map per domain.
3	Unknown Unknowns	1.5 days	NEW. "We have insufficient evidence for Executive hiring, Compensation, Board decisions." Honest gaps.
4	Organizational Curiosity	2 days	Maestro asks the org questions. "Was Alice's departure planned?" Feedback partnership.
5	Understanding Map	2 days	NEW. Visual coverage that grows with each recommendation. Customers see the product compound.
6	Unified "Your Organization" Surface	3 days	Merges V6+V7 into one screen. One page. Everything else command palette.
TOTAL 6 V8 specs		~12 days	V8: from 19 engines to 1 product — Institutional Memory

SPEC #1

Institutional Memory Timeline — experiential moments, not literary chapters

V8 Principle

V8: "Organizations don't read autobiographies. They revisit moments. Memory should feel experiential, not literary." Instead of "2025 Q1: Your organization became cautious," show a moment card: "March 14: Deployment failure. Maestro predicted 62%. Reality: failure occurred. Learning: review order mattered more than deployment size. Institution changed. Every deployment recommendation since then uses this rule."

Current codebase gap

evolution_narrative.py (162 lines) produces 5 literary chapters ("Who you are," "What you've learned"). The autobiography surface (autobiography.js, 41 lines) renders them as a narrative. The gap: the narrative is literary, not experiential. Organizations do not read autobiographies. They revisit moments — specific predictions, specific outcomes, specific learnings, specific institutional changes.

Files to create/modify

CREATE backend/maestro_oem/memory_timeline.py — the MemoryTimelineEngine. Produces TIMELINE MOMENTS, not chapters. Each moment is a card: {date, event, prediction (what Maestro predicted), reality (what happened), learning (what the org learned), institution_change (how the org changed), evidence_count}. Sources: prediction_lifecycle.py (resolved predictions — each is a moment), evolution_tracker.py (eliminated failure modes — each is a moment), model_revision.py (V7.1, when built — each revision is a moment). REPLACE static/js/autobiography.js with static/js/memory_timeline.js — renders moments as a vertical timeline of cards (not paragraphs). Each card is self-contained: date + event + prediction + reality + learning + change. Modify the command-palette entry from "Your Organization's Story" to "What we've learned."

API contract

GET /api/oem/memory-timeline returns: { "moments": [{ "date": "2024-11-12", "event": "payments-edge circuit breaker PR", "prediction": "Maestro predicted 62% success", "reality": "Failure occurred — circuit breaker did not trigger in time", "learning": "Review order mattered more than deployment size. The PR was reviewed after merge, not before.", "institution_change": "Every deployment recommendation since Nov 12 now requires pre-merge review for circuit-breaker changes.", "evidence_count": 3 }], "summary": "Your organization has 12 learning moments. The most recent: a deployment failure that changed how circuit-breaker reviews work." }. Must have at least 3 moments with all 6 fields (or honest "insufficient history").

Acceptance test

1) GET /api/oem/memory-timeline returns moments array (3+ or honest). 2) Each moment has date, event, prediction, reality, learning, institution_change. 3) Moments reference real prediction_lifecycle / evolution_tracker data. 4) The "What we've learned" surface renders moments as cards (not paragraphs). 5) V5 litmus: no new sidebar item (command-palette only). 6) V8 litmus: does this make the customer say "Maestro knows how our company works"? YES — it shows specific moments where Maestro learned.

Effort

2 days (1.5 days engine + 0.5 day frontend)

Dependencies

prediction_lifecycle.py (for resolved predictions). evolution_tracker.py (for eliminated failure modes). Replaces evolution_narrative.py + autobiography.js.

SPEC #2

Institutional Confidence — "How well do we understand Finance? 96%"

V8 Principle

V8: "Not prediction confidence. Institutional confidence. Imagine the CEO asks 'Should I trust Maestro about Marketing?' Maestro answers: 'No. We've only observed 14 campaigns. Our understanding is still immature.' That honesty becomes an enormous trust builder."

Current codebase gap

No module tracks per-domain understanding coverage. confidence.py tracks per-LAW confidence (how sure am I about this pattern?). But there is no per-DOMAIN confidence (how well do I understand the Finance domain? The Engineering domain?

The Sales domain?). The gap: Maestro cannot say "I understand Engineering at 91% but Sales at 18%" — it has no domain-level coverage metric.

Files to create/modify

CREATE `backend/maestro_oem/institutional_confidence.py` — the InstitutionalConfidenceEngine. For each organizational domain (from signal metadata — payments, auth, platform, deployment, legal, etc.): compute `coverage_score` (0-100) based on: (1) `signal_count` in that domain, (2) `learning_objects` in that domain, (3) validated laws in that domain, (4) resolved predictions about that domain, (5) causal chains involving that domain. Also compute `maturity_label` (nascent/developing/mature/deep). CREATE GET `/api/oem/confidence` endpoint. MODIFY `static/js/today.js` — add an "Understanding" section: a calm coverage display per domain (not a chart — a sentence: "We understand Engineering deeply (91%). We're still learning about Sales (18%).")

API contract

GET `/api/oem/confidence` returns: { "domains": [{ "domain": "payments", "confidence": 0.91, "maturity": "deep", "signal_count": 12, "laws": 3, "resolved_predictions": 5, "narrative": "We understand payments deeply. 12 signals, 3 validated patterns, 5 resolved predictions." }, { "domain": "sales", "confidence": 0.18, "maturity": "nascent", "signal_count": 2, "laws": 0, "resolved_predictions": 0, "narrative": "We're just starting to understand sales. 2 signals, no patterns yet. Trust our recommendations here with caution." }], "overall_confidence": 0.62, "narrative": "We understand payments and auth deeply. We're still learning about sales and legal. Trust our recommendations accordingly." }. Must have at least 3 domains with confidence + maturity + narrative.

Acceptance test

1) GET `/api/oem/confidence` returns domains array (3+) each with confidence (0-1), maturity, `signal_count`, narrative. 2) Low-confidence domains have honest narrative ("Trust with caution"). 3) TODAY shows "Understanding" section with per-domain confidence. 4) V5 litmus: no new panel — enhances TODAY. 5) V8 litmus: does this make the customer trust Maestro MORE? YES — honesty about gaps builds trust.

Effort

1.5 days (1 day coverage-scoring engine + 0.5 day API + frontend)

Dependencies

All existing modules (signal counts per domain, learning objects, laws, predictions, causal chains). No new dependencies.

SPEC #3

Unknown Unknowns — "We have insufficient evidence for Executive hiring"

V8 Principle

V8: "Today Maestro reports known patterns. I want areas we do not understand. 'We have insufficient evidence for Executive hiring, Compensation, Board decisions, Vendor negotiations, International expansion.' That is scientifically honest. And it naturally drives adoption."

Current codebase gap

curiosity.py (184 lines) identifies untested assumptions and unmeasured domains. But it frames them as QUESTIONS ("Nobody has tested whether..."). V8 reframes them as GAPS: areas where Maestro has NO understanding. The gap: no module explicitly lists what Maestro does NOT know — the blind spots, the coverage holes, the domains with zero signals.

Files to create/modify

CREATE `backend/maestro_oem/unknown_unknowns.py` — the UnknownUnknownsEngine. Produces a list of organizational domains/activities where Maestro has insufficient evidence (< 3 signals or < 1 learning object). Sources: scan all signal types in `signal_classes.py`, identify which organizational activities (hiring, compensation, board decisions, vendor negotiations, international expansion, M&A, budgeting, performance reviews, etc.) have ZERO or near-zero signal coverage. CREATE GET `/api/oem/unknowns` endpoint. MODIFY `static/js/today.js` — add an "What Maestro doesn't know yet" section: "We don't understand: Executive hiring (0 signals), Compensation (0 signals), Board decisions (0 signals). Connect more signal sources to learn about these areas."

API contract

GET `/api/oem/unknowns` returns: { "blind_spots": [{ "area": "Executive hiring", "signal_count": 0, "reason": "No HR or recruiting signals ingested. Connect an HR system to learn about hiring decisions.", "impact": "Maestro cannot advise on hiring patterns or departure risks at the executive level." }, { "area": "Compensation", "signal_count": 0, "reason": "No compensation or payroll signals. Compensation decisions are invisible to Maestro.", "impact": "Maestro cannot detect compensation-related retention risks." }], "coverage_gaps": [{ "area": "Sales", "signal_count": 2, "reason": "Only 2 signals from sales. Insufficient for pattern detection.", "impact": "Sales recommendations are low-confidence." }], "narrative": "Maestro has 5 complete blind spots and 2 coverage gaps. Connecting HR, compensation, and sales systems would significantly improve understanding." }. Must have at least 3 blind_spots with area + reason + impact.

Acceptance test

1) GET `/api/oem/unknowns` returns blind_spots (3+) + coverage_gaps. 2) Each blind_spot has area, signal_count (0 or near-0), reason, impact. 3) TODAY shows "What Maestro doesn't know yet" section. 4) V5 litmus: no new panel — enhances TODAY. 5) V8 litmus: does this build trust? YES — honesty about gaps is the strongest trust signal.

Effort

1.5 days (1 day gap-detection engine + 0.5 day API + frontend)

Dependencies

`signal_classes.py` (to enumerate signal types). `institutional_confidence.py` (V8 #2, for coverage thresholds). `curiosity.py` (for existing untested-assumption logic).

SPEC #4

Organizational Curiosity — Maestro asks the org questions

V8 Principle

V8: "Instead of waiting for signals, Maestro starts asking: 'We don't understand why approvals doubled this week. Can you tell us?' or 'Was Alice's departure planned?' or 'Did Legal change policy?' The organization teaches Maestro. Maestro teaches the organization. That's a genuine feedback partnership."

Current codebase gap

curiosity.py (184 lines) generates questions for the user ("Nobody has tested whether..."). But these are passive — they appear in TODAY and wait for the user to act. V8 makes curiosity ACTIVE: Maestro detects anomalies (approval rate doubled, signal pattern changed, a key person disappeared from signals) and ASKS the organization directly. The gap: no mechanism for Maestro to ask a question, receive an answer, and incorporate the answer into its model.

Files to create/modify

CREATE backend/maestro_oem/organizational_curiosity.py — the OrganizationalCuriosityEngine. Detects anomalies (from signal history: "approval count doubled this week vs 4-week average"), generates specific questions ("We noticed approval volume doubled this week. Was this intentional?"), and provides a mechanism for the user to ANSWER (text input or multiple choice). Each answer is stored as a new signal (type: human_context) that feeds into the model. CREATE GET /api/oem/curiosity/questions endpoint (returns open questions). CREATE POST /api/oem/curiosity/answer endpoint (accepts answer, creates signal). MODIFY static/js/today.js — add a "Maestro has questions" card: "We noticed approval volume doubled this week. Was this intentional?" with a text input + submit button. When answered, the card disappears and the answer becomes a signal.

API contract

GET /api/oem/curiosity/questions returns: { "questions": [{ "id": "q-001", "question": "We noticed approval volume doubled this week (from 3 to 7). Was this intentional?", "context": "4-week average was 3.2 approvals/week. This week: 7.", "type": "anomaly_explanation", "asked_at": "..."}], "open_count": 1 }. POST /api/oem/curiosity/answer accepts: { "question_id": "q-001", "answer": "Yes, we're catching up after the holiday backlog." }. Returns: { "ok": true, "signal_created": true, "signal_id": "sig-xxx" }. The answer becomes a human_context signal that feeds into the model.

Acceptance test

1) GET /api/oem/curiosity/questions returns questions array (1+ or honest "no open questions"). 2) Each question has id, question, context, type. 3) POST /api/oem/curiosity/answer creates a signal (verified: signal appears in model after answering). 4) TODAY shows "Maestro has questions" card with input + submit. 5) After answering, card disappears. 6) V5 litmus: no new panel — enhances TODAY. 7) V8 litmus: does this make the customer say "Maestro knows how our company works"? YES — the org teaches Maestro directly.

Effort

2 days (1.5 days anomaly detection + question generation + answer ingestion + 0.5 day API + frontend)

Dependencies

signal history (for anomaly detection). signal.py (for creating human_context signals). curiosity.py (for existing question logic).

SPEC #5

Understanding Map — visual coverage that grows with each recommendation

V8 Principle

V8: "Imagine every recommendation literally coloring in a map of organizational understanding. Engineering: 91%. Sales: 18%. Customers would immediately understand why the product compounds." Every recommendation should grow the map. After each resolved prediction, the domain's coverage increases.

Current codebase gap

institutional_confidence.py (V8 #2) computes per-domain confidence. But it is a number, not a visual. The gap: no visual representation of coverage that makes the compounding visible. A CEO should be able to SEE that Maestro understands Engineering deeply (green bar) but Sales poorly (red bar) — and that the green bars are growing over time.

Files to create/modify

CREATE `backend/maestro_oem/understanding_map.py` — the UnderstandingMapEngine. Produces a visual map: for each domain, a coverage bar (0-100%) with color (red < 30%, amber 30-60%, green 60-90%, deep green > 90%). Also produces a GROWTH RATE: "Engineering coverage grew 12% this month (from 79% to 91%)." The map updates after each resolved prediction (the domain's coverage increases). CREATE `GET /api/oem/understanding-map` endpoint. CREATE `static/js/understanding_map.js` — renders the map as horizontal bars (one per domain) with color + percentage + growth indicator. MODIFY `static/js/learn.js` — add the Understanding Map as the TOP section (before everything else): the first thing the user sees in LEARN is "How well Maestro understands your organization."

API contract

`GET /api/oem/understanding-map` returns: { "domains": [{ "domain": "payments", "coverage": 91, "color": "deep_green", "growth_this_month": "+3%", "narrative": "Payments: deeply understood (91%). Growing steadily." }, { "domain": "auth", "coverage": 78, "color": "green", "growth_this_month": "+5%", "narrative": "Auth: well understood (78%). Fast growth." }, { "domain": "sales", "coverage": 18, "color": "red", "growth_this_month": "0%", "narrative": "Sales: barely understood (18%). No growth. Connect sales signals." }], "overall_coverage": 62, "overall_growth": "+4% this month", "narrative": "Maestro understands 62% of your organization. Coverage grew 4% this month, mostly from resolved predictions in auth and payments." }. Must have at least 3 domains with coverage + color + growth + narrative.

Acceptance test

1) `GET /api/oem/understanding-map` returns domains (3+) with coverage (0-100), color, growth_this_month, narrative. 2) LEARN surface shows Understanding Map as TOP section. 3) Map is visual (horizontal bars with colors). 4) growth_this_month is computed (not hardcoded). 5) V5 litmus: enhances LEARN (no new panel). 6) V8 litmus: does this make the customer say "Maestro knows how our company works"? YES — the map VISUALLY proves it.

Effort

2 days (1 day map engine + growth tracking + 0.5 day API + 0.5 day frontend visualization)

Dependencies

institutional_confidence.py (V8 #2). prediction_lifecycle.py (for growth tracking — each resolved prediction grows the domain).

SPEC #6

Unified "Your Organization" Surface — one page, everything else command palette

V8 Principle

V8: "Eventually the product becomes: Your Organization — Today / Needs attention / Learning / Recent changes / Understanding / Open questions / Prepared decisions. One page. Everything else Command Palette." The customer never sees 19 engines. They see one page that answers "What have we learned?"

Current codebase gap

TODAY, LEARN, and Cognition are 3 separate surfaces with overlapping content. TODAY has the morning brief + nudges + curiosity + background loop + trajectory intervention. LEARN has DNA + evolution tracker + understanding map (V8 #5). Cognition has 10+ organ render sections. The gap: the customer must navigate between 3 surfaces to understand "what we've learned." V8 unifies them into one "Your Organization" surface.

Files to create/modify

RESTRUCTURE static/js/today.js — rename to "Your Organization" surface. Merge the key content from LEARN and Cognition into a SINGLE unified page with 7 sections: (1) Today (morning brief — current), (2) Needs attention (trajectory intervention + nudges — current), (3) Learning (curiosity questions + background notices — current), (4) Recent changes (evolution tracker + DNA evolution — from LEARN), (5) Understanding (understanding map + institutional confidence — from V8 #2/#5), (6) Open questions (curiosity questions — from V8 #4), (7) Prepared decisions (executive function plans — current). MOVE Cognition surface to command-palette only (it becomes a "deep view" for power users, not the default). Sidebar: "Today" becomes "Your Organization" (or just "Maestro"). LEARN and Cognition remain accessible via command palette but are NOT in the sidebar. Sidebar: "Your Organization" + "Work" + "Ask" + "More..." = 4 items (unchanged).

API contract

No new API. This is a frontend restructure. The unified surface fetches from: /ceo-briefing, /nudges, /curiosity/questions, /background-loop, /trajectory-intervention, /evolution-tracker, /dna, /understanding-map, /confidence, /execute. All existing APIs. The restructure is in how the content is PRESENTED — one page, 7 sections, scrollable, calm.

Acceptance test

1) The "Your Organization" surface (renamed from TODAY) contains 7 sections. 2) Each section renders content from existing APIs (no new APIs). 3) LEARN and Cognition are command-palette only (not in sidebar). 4) Sidebar: 4 items ("Your Organization" + "Work" + "Ask" + "More..."). 5) V5 litmus: SIMPLER (3 surfaces merged into 1). 6) V8 litmus: does this make the customer say "Maestro knows how our company works"? YES — one page answers everything.

Effort

3 days (2 days frontend restructure + 0.5 day API consolidation + 0.5 day testing)

Dependencies

All V3-V6 APIs (existing). V8 #1-#5 (for timeline, confidence, unknowns, curiosity, map content).

Build Order

Step	Spec	Effort	Dependency	Key Output
0	Fix V6 wiring gaps	1.5 hours	None	Background loop in live_ingest + DNA in wisdom.py
1	#1 Institutional Memory Timeline	2 days	prediction_lifecycle.py + evolution_tracker.py	Key historical moments (not literary chapters)
2	#2 Institutional Confidence	1.5 days	All existing modules	Per-domain coverage score ("Finance: 96%")
3	#3 Unknown Unknowns	1.5 days	V8 #2 (for coverage thresholds)	Honest gaps ("We don't understand: hiring, compensation")
4	#4 Organizational Curiosity	2 days	signal history + signal.py	Maestro asks the org questions + ingests answers
5	#5 Understanding Map	2 days	V8 #2 (institutional confidence)	Visual coverage bars that grow over time
6	#6 Unified Surface	3 days	V8 #1-#5 + all V3-V6 APIs	One page. Everything else command palette.
TOTAL	6 V8 specs + V6 fixes	~12 days + 1.5 hours	V3-V6 complete	V8: from 19 engines to 1 product — Institutional Memory

The V8 Litmus Test

DOES THIS MAKE THE CUSTOMER SAY "MAESTRO KNOWS HOW OUR COMPANY WORKS"?

V5 litmus: "Is the UI simpler?" (retained).

V6 litmus: "Does this permanently improve the organization?" (retained).

V7 litmus: "Does this make Maestro's understanding more accurate?" (retained, when V7.1 is built).

V8 litmus (NEW): "Does this make the customer say 'Maestro knows how our company works'?"

Every V8 spec must pass. The bar is not "does the engine work?" or "is the UI simpler?" or "does the org improve?" It is: does the CUSTOMER experience Maestro as something that knows their company? If the customer sees engine names, version numbers, or implementation details, it fails. If the customer sees "What we've learned" and feels understood, it passes.

The End State

"MAESTRO KNOWS HOW OUR COMPANY WORKS"

One year later, the customer no longer says "We use Maestro." They say "Maestro knows how our company works." That sentence is worth more than any feature, any dashboard, any copilot. Because nobody else can say it.

The 19 engines remain internally. The customer sees one product: Institutional Memory. They open one page: "Your Organization." They see: what needs attention, what they've learned, where they're drifting, what Maestro doesn't know yet, what questions Maestro has for them, and how well Maestro understands each part of the company. They never see "DNA" or "Institution Model" or "Trajectory Intervention." They see understanding.

The constitutional law: "Maestro should become the living institutional memory of an organization — continuously observing, questioning, revising, and deepening its understanding until it becomes the most accurate model of how that organization thinks, decides, and learns."

Build order: Step 0 (fix V6 gaps, 1.5 hours) -> #1 Timeline (2 days) -> #2 Confidence (1.5 days) -> #3 Unknowns (1.5 days) -> #4 Curiosity (2 days) -> #5 Map (2 days) -> #6 Unified Surface (3 days). Total ~12 days + 1.5 hours. When complete, V8 is done. The product is Institutional Memory. Ship the 90-day pilot.